



大规模信息系统构建技术导论

分布式 MiniSQL 系统设计报告

小组成员

学号	姓名
组长信息	刘星宇
组员信息	邹明昊

2026 年 5 月 5 日

目录

一．系统简介	3
1.1 系统目标	3
1.2 设计说明	3
二．总体设计	3
2.1 总体架构	4
2.2 流程图设计	4
三．详细设计	5
3.1 分布式架构	5
3.1.1 数据分片设计	5
3.1.2 副本架构设计	5
3.1.3 元数据设计	6
3.1.4 分布式查询设计	6
3.2 集群管理	7
3.2.1 zookeeper 协调服务设计模块	7
3.2.2 集群管理模块	8
3.3 容错容灾	10
3.4 负载均衡	11
3.5 客户端	11
四．测试用例设计	12
4.1 SQL 功能测试	12
4.2 集群管理	14
4.3 容错容灾	14
4.4 负载均衡	15
4.5 客户端	15

一. 系统简介

本项目是《大规模信息系统构建技术导论》的课程项目。目的是完成一个分布式的miniSQL系统，可以完成一个分布式的关系型数据库的各类基本操作，SQL语的执行，并且具有副本维护、负载均衡、容错容灾等功能

与单机 MiniSQL 相比，分布式 MiniSQL 的主要改进包括：第一，数据可以按照分片键分布到多个数据节点，突破单机容量限制；第二，每个分片可以维护多个副本，提升系统可用性；第三，查询请求可以被拆分为多个子任务并行执行，提高查询吞吐；第四，系统能够通过心跳检测、故障切换和副本恢复机制在节点异常时继续提供服务

1.1 系统目标

该分布式 MiniSQL具体功能如下：

- (1) 基础 SQL 功能：包括 CREATE TABLE、DROP TABLE、INSERT、DELETE、UPDATE 和 SELECT
- (2) 数据分片：建表时可以指定分片键，系统根据哈希分片规则将数据分布到多个 DataNode
- (3) 分布式查询：Coordinator 根据查询条件进行分片裁剪或广播查询，并将子查询并行下发到多个数据节点执行
- (4) 复杂查询：系统至少支持内连接 Join，优先实现同分片 Join 和广播 Join
- (5) 副本管理：每个分片采用 1 主 2 从的副本策略，写请求进入主副本，再同步或半同步复制到从副本
- (6) 集群管理：系统维护节点注册、节点下线、心跳检测、节点状态和集群拓扑
- (7) 容错容灾：主副本故障后，系统能够从健康从副本中选择新主，并更新元数据路由
- (8) 负载均衡：读请求可以在多个健康副本之间分配，降低单个节点压力
- (9) 分布式协调：系统使用 ZooKeeper 维护临时节点、监听节点上下线、并通知路由版本变化

1.2 设计说明

本大程采用 Java 程序设计语言，在IDEA、VSCODE等集成开发环境中编辑、编译与调试。具体程序由 2 人组成的小组开发而成。小组成员的具体分工如表 1 所示：

表 1 各成员分工表

成员姓名	学号	分工
刘星宇	3230106036	组长。负责总体架构、元数据管理、DataNode、本地执行适配、集群管理、模块集成和验收材料整理
邹明昊	3230105159	负责 SQL 解析、分布式查询计划、路由模块、副本管理、容错恢复、测试脚本

二. 总体设计

本系统主要由 1 个 Coordinator、1 个 MetaStore 和 3 个 DataNode，以及辅助功能模块Replica Manager（副本管理模块）、Cluster Manager（集群管理模块）、Failover Manager（容灾模块）、Load Balancer（负载

均衡模块)

2.1 总体架构

(1) Client

客户端提供字符界面，允许用户输入 SQL 语句，显示查询结果、影响行数、错误信息和执行耗时。客户端不直接访问数据节点，而是统一连接 Coordinator

(2) Coordinator

Coordinator 是系统入口，负责 SQL 接入、语法解析、语义检查、执行计划生成、分片路由、任务调度、结果合并和错误处理。Coordinator 内部包含 Parser、Analyzer、Planner、Router、Scheduler 和 Result Merger 等子模块

(3) MetaStore/Catalog Manager

MetaStore 维护全局元数据，包括表结构、字段信息、索引信息、分片规则、分片到节点的映射、副本角色和路由版本。Coordinator 在执行 SQL 前通过 MetaStore 获取或刷新路由信息。节点存活状态优先由 ZooKeeper 临时节点和 watch 机制感知

(4) DataNode

DataNode 负责保存实际分片数据，并调用本地 MiniSQL 执行子查询和写入操作。每个 DataNode 可以承载多个分片的主副本或从副本，并向 Coordinator 上报心跳和负载

(5) zookeeper

ZooKeeper 是外部协调服务，提供节点存活检测、watch 通知和锁服务

(6) Replica Manager

Replica Manager 负责主从复制日志的生成、传输、回放和进度记录。写请求先进入主副本，主副本生成日志后复制到从副本

(7) Cluster Manager

Cluster Manager 基于 ZooKeeper 模块完成节点注册、节点下线、心跳检测、状态转换和拓扑维护。节点状态包括 **ONLINE**、**SUSPECT**、**OFFLINE** 和 **RECOVERING**

(8) Failover Manager

Failover Manager 在 ZooKeeper watch 到主副本节点失效后，从健康从副本中选择复制进度最新的节点提升为新主，并通过 ZooKeeper 分布式锁避免多个 Coordinator 同时切主

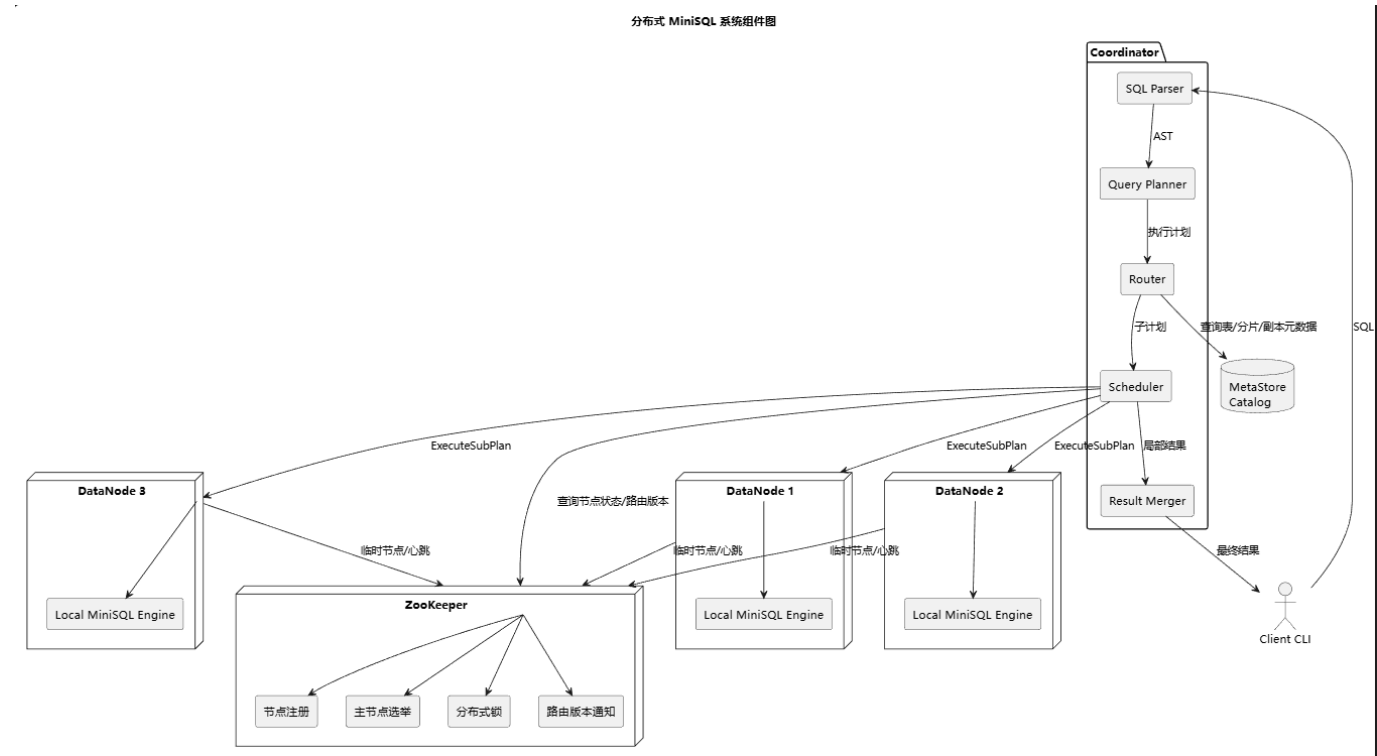
(9) Load Balancer

Load Balancer 根据节点状态、CPU、QPS、平均延迟和副本延迟等指标，在健康副本之间选择合适的读请求目标

2.2 流程图设计

系统主要执行流程如下：

分布式 MiniSQL 系统组件图



三. 详细设计

3.1 分布式架构

3.1.1 数据分片设计

系统采用哈希分片：

```
shard_id = hash(shard_key) % shard_count
```

建表时指定分片键和分片数量。例如：

```
CREATE TABLE student (  
    sid INT PRIMARY KEY,  
    name CHAR(20),  
    age INT,  
    dept CHAR(20)  
) SHARD BY HASH(sid) SHARDS 3 REPLICAS 3;
```

3.1.2 副本架构设计

多副本管理采用主从架构。每个分片维护 1 个主副本和 2 个从副本。写请求只进入主副本，从副本通过复制日志保持数据一致

示例副本布局如下：

分片	主副本	从副本 1	从副本 2
----	-----	-------	-------

分片	主副本	从副本 1	从副本 2
shard_0	dn1	dn2	dn3
shard_1	dn2	dn1	dn3
shard_2	dn3	dn1	dn2

这种交错布局可以使主副本均匀分布在多个节点上，避免所有写压力集中在单个节点

3.1.3 元数据设计

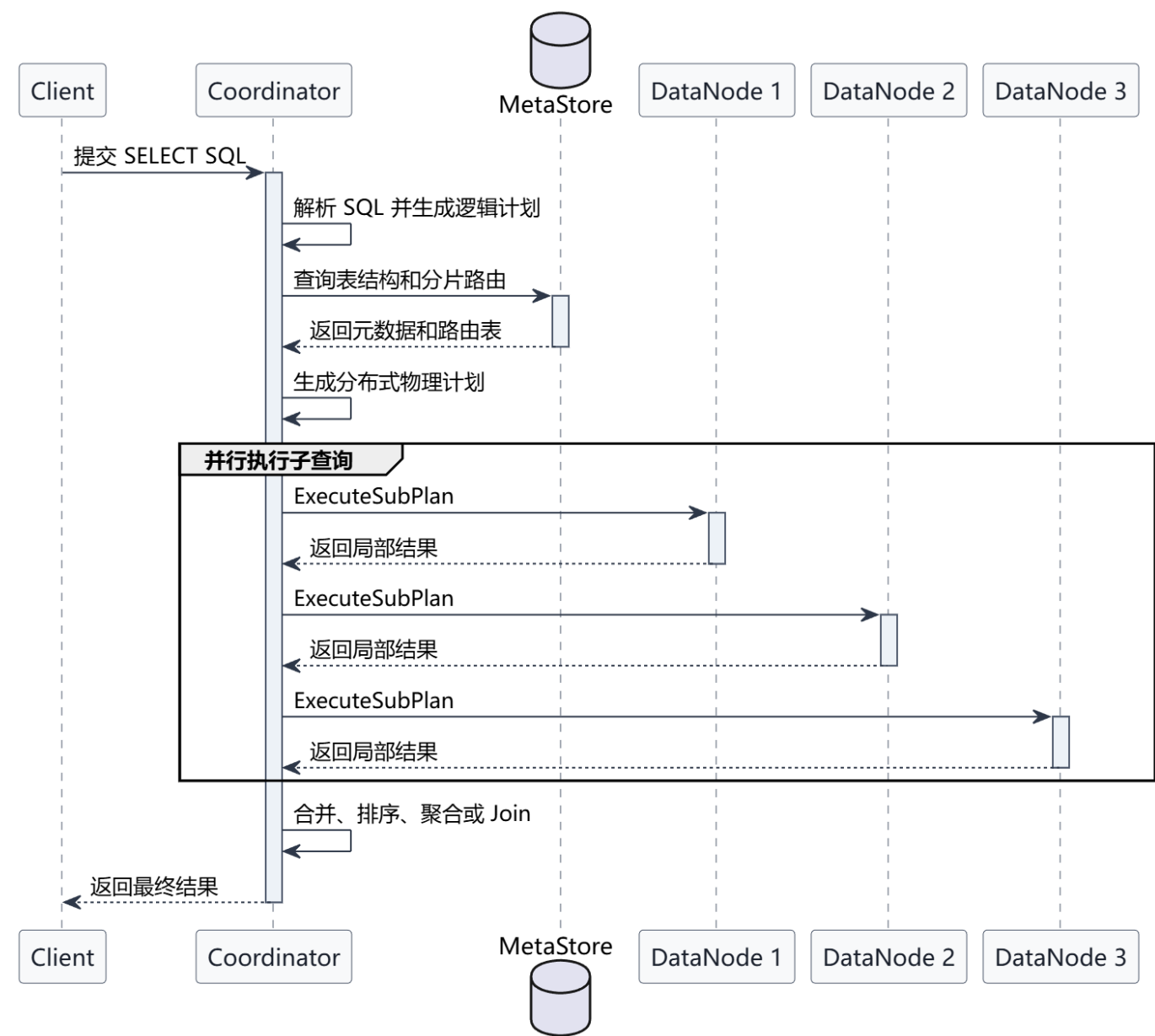
MetaStore 保存以下元数据：

元数据表	关键字段	作用
tables	table_id, table_name, schema_version, shard_key	表定义
columns	table_id, column_name, type, nullable, is_primary	字段定义
shards	shard_id, table_id, rule_type, rule_value	分片定义
nodes	node_id, host, port, status, last_heartbeat	节点信息
route_versions	version, updated_at, description	路由版本，MetaStore 中路由版本变化后，Coordinator 刷新缓存，保证请求能够被路由到最新主副本

3.1.4 分布式查询设计

以数据库的查询功能为例，查询处理分为 SQL 解析、语义检查、逻辑计划生成、物理计划生成和分布式调度五个步骤，利用MapReduce的思想，先分再合

时序图如下：



3.2 集群管理

本系统使用zookeeper和cluster manager模块一起完成集群的管理

3.2.1 ZooKeeper 协调服务设计模块

ZooKeeper 在本系统中用于保存少量关键协调信息，不保存用户表数据。业务数据仍然存储在 DataNode 的本地 MiniSQL 存储层中

ZooKeeper 节点目录设计如下：

```
/minisql
/coordinators
  /candidate-000000001
  /candidate-000000002
/leader
/datanodes
```

```

    /dn1
    /dn2
    /dn3
  /locks
    /failover
      /shard_0
      /shard_1
  /routes
    /version
  /shards
    /student
      /shard_0
      /shard_1
      /shard_2

```

各目录作用如下：

路径	类型	作用
/minisql/coordinators	临时顺序节点	多 Coordinator 部署时进行主节点选举
/minisql/leader	临时节点	记录当前主 Coordinator
/minisql/datanodes/dnX	临时节点	DataNode 注册和故障感知
/minisql/locks/failover/shard_X	临时节点	分片故障切换锁
/minisql/routes/version	持久节点	路由版本号，变更时触发 watch
/minisql/shards/...	持久节点	可保存轻量分片状态，完整元数据仍以 MetaStore 为准

ZooKeeper 的使用场景如下：

- (1) **DataNode 注册**：DataNode 启动后创建 /minisql/datanodes/dnX 临时节点，节点内容包含地址、端口、启动时间和当前负载摘要。DataNode 进程异常退出或网络断开后，临时节点自动消失
- (2) **故障感知**：Coordinator 对 /minisql/datanodes 设置 watch。当某个 DataNode 临时节点消失时，Coordinator 触发故障判断和副本切换流程
- (3) **Coordinator 选主**：多个 Coordinator 启动时在 /minisql/coordinators 下创建临时顺序节点，编号最小者成为主 Coordinator。只有主 Coordinator 执行写元数据、故障切换和分片迁移
- (4) **分布式锁**：Failover Manager 在切换某个分片主副本前，先尝试创建 /minisql/locks/failover/shard_X 临时节点。创建成功者获得锁，其他 Coordinator 放弃或等待，避免重复切主
- (5) **路由版本通知**：MetaStore 中分片布局或副本角色变化后，Coordinator 更新 ZooKeeper 中 /minisql/routes/version。其他 Coordinator 或客户端代理监听到版本变化后刷新本地路由缓存

3.2.2 集群管理模块

集群管理模块负责维护 DataNode 的生命周期，包括节点注册、心跳检测、状态转换和拓扑更新

节点状态机如下：

```

    ONLINE -> SUSPECT -> OFFLINE -> RECOVERING -> ONLINE

```

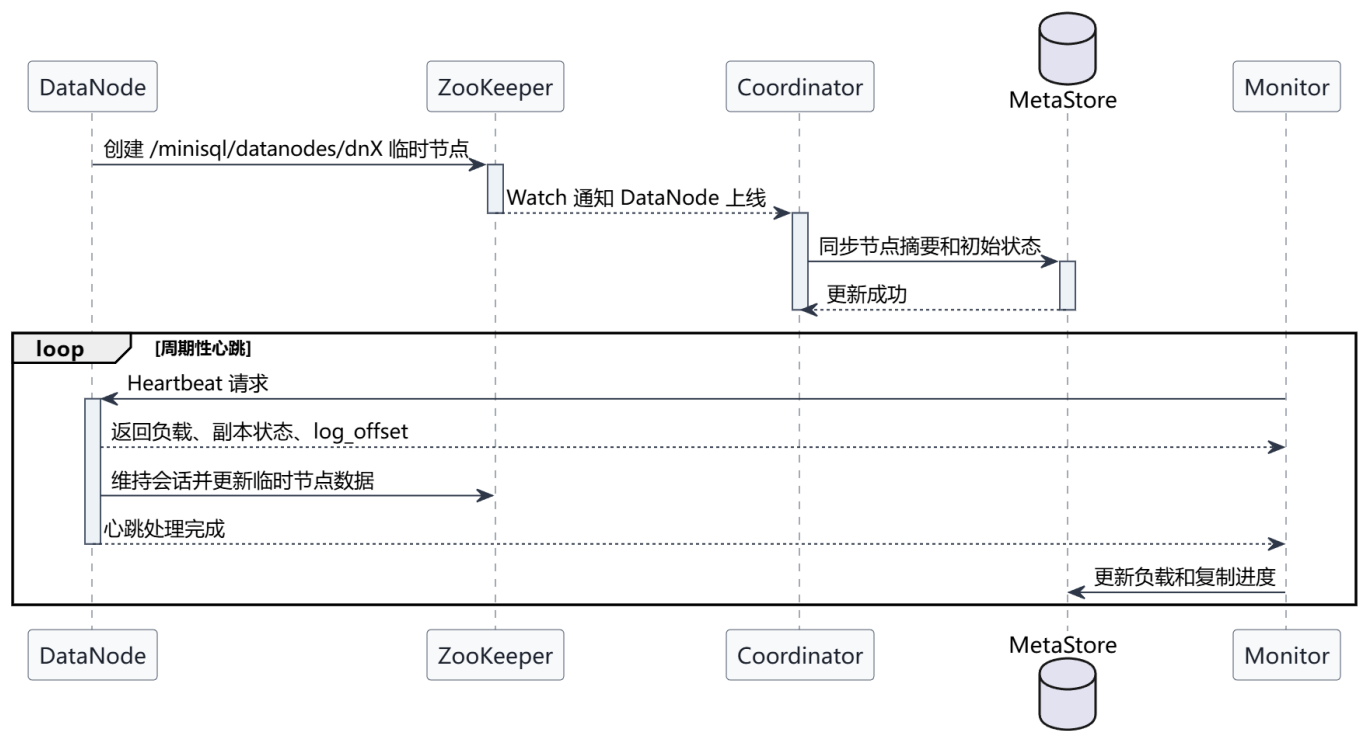
- **ONLINE** ： 节点正常，可承担读写请求
- **SUSPECT** ： 心跳短暂异常，暂不分配新请求
- **OFFLINE** ： 节点确认故障，需要触发故障恢复
- **RECOVERING** ： 节点恢复中，正在补齐数据

DataNode 启动后调用 ZooKeeper 注册临时节点，注册信息包括节点编号、地址、端口和可用存储目录。
Coordinator 监听 ZooKeeper 中的 DataNode 子节点变化，并将节点摘要同步到 MetaStore 或内存状态表

心跳/事件	Cluster Manager 处理
ONLINE	更新负载和日志位点；必要时从 SUSPECT 恢复
SUSPECT	暂停新请求调度，继续观察
OFFLINE	移出调度；若承载主副本则触发切主
RECOVERING	只允许日志补齐；补齐后转 ONLINE
ZooKeeper 临时节点消失	直接标记 OFFLINE，触发故障判断

为减少误判，系统采用连续超时机制。例如心跳周期为 2 秒，连续 3 次超时才将节点从 **SUSPECT** 标记为 **OFFLINE**

集群管理的时序图如下：



3.3 容错容灾

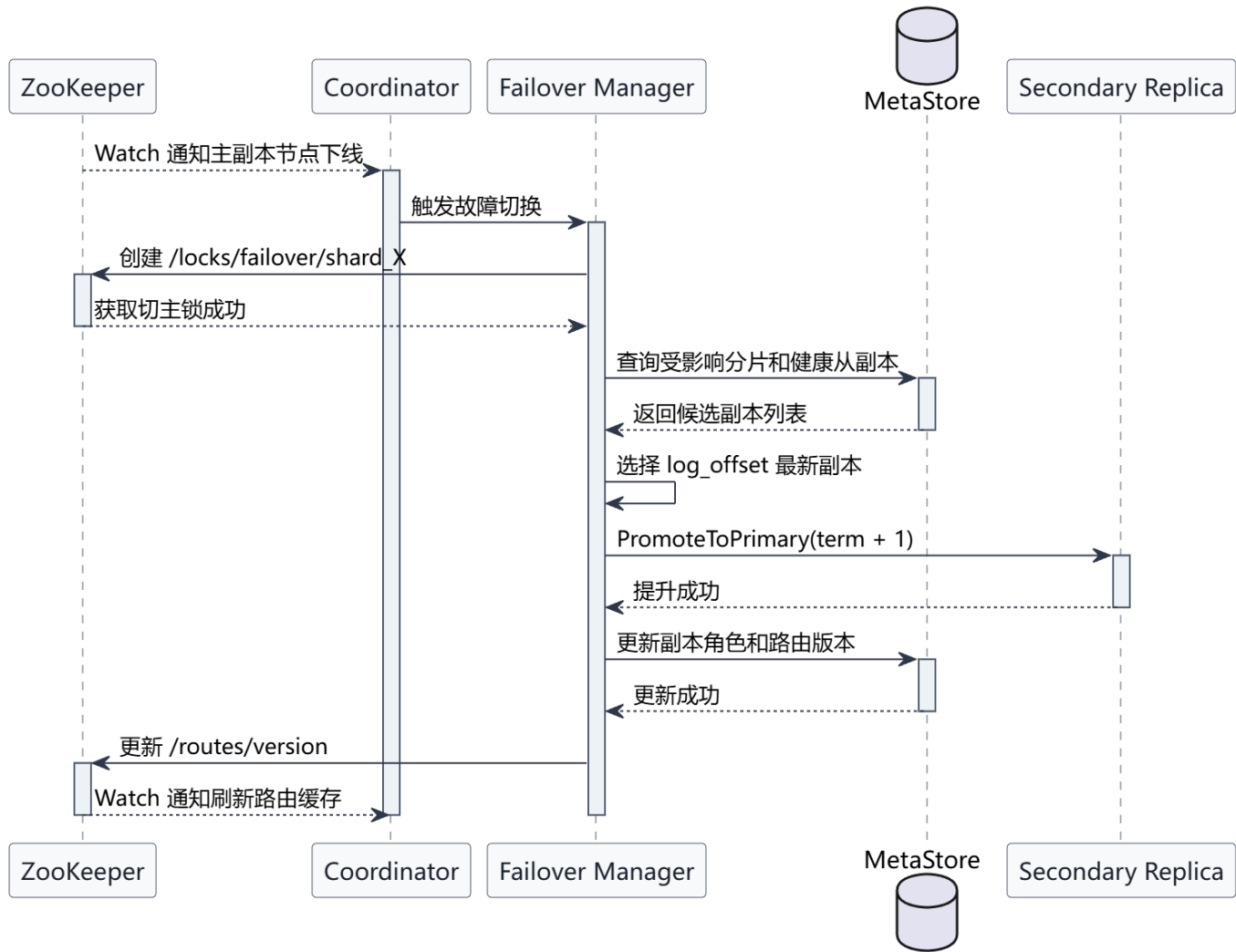
容错容灾模块主要处理 DataNode 故障、主副本切换和恢复节点数据补齐。节点故障后，Coordinator/Failover Manager 先修复元数据和主副本角色，再由健康 DataNode 接管服务。

当某个 DataNode 故障时，系统执行以下步骤：

- (1) DataNode 与 ZooKeeper 的会话失效，对应临时节点自动删除
- (2) Coordinator 通过 watch 感知节点下线，Cluster Manager将节点标记为 **OFFLINE**
- (3) Failover Manager 查询 MetaStore，找出该节点承载主副本的所有分片
- (4) 对每个受影响分片，先通过 ZooKeeper 获取分片切主锁
- (5) 从健康从副本中选择 **log_offset** 最新的副本
- (6) 将选中的从副本提升为新主，并递增副本任期 **term**
- (7) 更新 MetaStore 中的 **replicas** 表和路由版本
- (8) 更新 ZooKeeper 中的 **/minisql/routes/version**，通知各组件刷新路由
- (9) 新主开始接收写请求
- (10) 原故障节点恢复后进入 **RECOVERING** 状态，从新主拉取缺失日志并重放成为副本

故障切换时需要防止双主问题。系统使用 **term** 表示副本任期，任期越大代表角色信息越新。旧主恢复后必须先向 MetaStore 查询最新任期和角色，不能直接提供写服务

主副本故障切换流程如下：



3.4 负载均衡

负载均衡模块主要负责读请求的副本选择和热点分片识别，读请求调度综合考虑以下指标：

- (1) 节点状态：只选择 **ONLINE** 节点
- (2) 副本角色：普通读可选择主副本或从副本
- (3) 副本延迟：过滤复制延迟超过阈值的从副本
- (4) 节点负载：根据 CPU、QPS、平均响应时间调整权重
- (5) 请求类型：强一致读优先访问主副本，普通读优先访问低负载从副本

采用加权轮询策略：

```
score = base_weight - cpu_weight - latency_weight - lag_weight
```

score越高，越优先承担读请求。对于热点分片，系统通过统计分片级 QPS 和平均响应时间进行识别。如果某个分片持续高于平均负载，系统会提高其从副本读权重，或在后续扩展中进行分片拆分和数据迁移

3.5 客户端

客户端通过字符界面与用户交互，允许用户输入 SQL 语句并查看执行结果。客户端功能包括：

- (1) 连接 Coordinator
- (2) 读取用户输入的 SQL，支持多行 SQL 输入，以分号作为结束标志
- (3) 显示查询结果表格、影响行数、错误信息和执行时间

四. 测试用例设计

4.1 SQL 功能测试

用例 1

考察分片表创建、元数据写入和副本布局生成

输入

```
CREATE TABLE student (  
  sid INT PRIMARY KEY,  
  name CHAR(20),  
  age INT,  
  dept CHAR(20)  
) SHARD BY HASH(sid) SHARDS 3 REPLICAS 3;
```

假设 ZooKeeper、Coordinator、MetaStore 和 3 个 DataNode 均正常运行

预期输出：建表成功，MetaStore 中生成表结构、分片规则、分片副本布局和路由版本。

后续用例默认已存在 student 表

用例 2

考察单条记录插入时是否能根据分片键写入正确分片主副本

输入

```
INSERT INTO student VALUES (1001, 'Alice', 20, 'CS');
```

预期输出：返回插入成功，记录写入 hash(1001) % 3 对应分片的主副本，并生成复制日志

用例 3

考察批量插入数据后，不同分片是否都有数据分布

输入

```
INSERT INTO student VALUES (1002, 'Bob', 21, 'Math');
INSERT INTO student VALUES (1003, 'Cindy', 19, 'EE');
INSERT INTO student VALUES (1004, 'David', 22, 'CS');
INSERT INTO student VALUES (1005, 'Eva', 20, 'Math');
```

预期输出：返回插入成功，各记录按照 $\text{hash}(\text{sid}) \% 3$ 分布到不同分片

用例 4

考察 int 类型分片键上的等值查询，以及 Coordinator 的分片裁剪能力

输入

```
SELECT * FROM student WHERE sid = 1001;
```

假设 sid 为 1001 的记录已经插入

预期输出：只访问目标分片，返回 sid = 1001 对应的完整记录

用例 5

考察 char 类型非分片键等值查询，以及广播查询结果合并

输入

```
SELECT sid, name FROM student WHERE dept = 'CS';
```

多个分片中可能存在 $\text{dept} = \text{'CS'}$ 的记录

预期输出：Coordinator 向所有相关分片下发子查询，合并后返回所有 CS 学生

用例 6

考察同分片 Join 或广播 Join 的查询结果正确性

输入

```
CREATE TABLE course (
  cid INT PRIMARY KEY,
  sid INT,
  cname CHAR(20)
) SHARD BY HASH(sid) SHARDS 3 REPLICAS 3;
INSERT INTO course VALUES (1, 1001, 'Database');
SELECT student.name, course.cname
FROM student JOIN course
```

```
ON student.sid = course.sid  
WHERE student.sid = 1001;
```

预期输出：返回正确的连接结果

4.2 集群管理

用例 7

考察 DataNode 注册、ZooKeeper 临时节点创建和集群状态显示

输入

```
SHOW CLUSTER;
```

假设 dn1、dn2、dn3 已依次启动并连接 ZooKeeper

预期输出：显示三个 DataNode 均为 **ONLINE**，并展示节点地址、端口、最近心跳时间和当前负载摘要

用例 8

考察节点心跳异常后的状态转换

暂停 dn2 心跳上报，分别等待 1 个和 3 个心跳周期后执行 SHOW CLUSTER

预期输出：dn2 先由 **ONLINE** 变为 **SUSPECT**，持续超时后变为 **OFFLINE**，Coordinator 不再向 dn2 分配新请求

用例 9

考察分片和副本布局展示

输入

```
SHOW SHARDS student;
```

假设 student 表已完成 3 分片、3 副本初始化

预期输出：显示 **shard_0**、**shard_1**、**shard_2** 的主副本、从副本、副本状态和路由版本

4.3 容错容灾

用例 10

考察主副本所在 DataNode 故障后的自动切主能力。

假设 shard_0 主副本位于 dn1，且在在 dn2 或 dn3 上至少存在一个健康从副本，人为停止 dn1，等待故障检测完成，之后执行

```
SHOW SHARDS student;
INSERT INTO student VALUES (1006, 'Frank', 21, 'EE');
```

预期输出：系统将健康从副本提升为新主，MetaStore 更新副本角色和路由版本，插入请求能够继续执行

用例 11

考察旧主恢复后的日志补齐和角色约束

恢复 dn1，之后执行

```
SHOW CLUSTER;
SHOW SHARDS student;
SELECT * FROM student WHERE sid = 1006;
```

预期输出：dn1 先进入 **RECOVERING** 状态，从新主拉取缺失日志，补齐后作为从副本回到 **ONLINE**，查询能够读到 sid 为 1006 的记录，且不能直接以旧主身份接受写请求

4.4 负载均衡

用例 12

考察普通读请求在多个健康副本之间的加权轮询分配

输入

```
SELECT * FROM student WHERE sid = 1001;
SELECT * FROM student WHERE sid = 1001;
SELECT * FROM student WHERE sid = 1001;
```

预期输出：多次普通读请求被分配到多个健康副本，单个节点不会长期承担全部读请求

4.5 客户端

用例 13

考察客户端连接 Coordinator 和合法 SQL 显示

启动客户端，连接 Coordinator，输入

```
SELECT * FROM student WHERE sid = 1001;
```

预期输出：客户端连接成功，查询结果以表格形式展示，并显示执行耗时

用例 14

考察语法错误 SQL 的错误提示

输入

```
SELEC * FORM student WHERE sid = 1001;
```

预期输出：返回明确的语法错误信息，包括错误类型、错误位置或错误 SQL 片段

用例 15

考察辅助命令展示执行计划和集群状态

输入

```
SHOW CLUSTER;  
SHOW SHARDS student;
```

预期结果：SHOW CLUSTER 展示节点状态；SHOW SHARDS student 展示分片副本布局